

Git Bash and PowerShell Command Cheat Sheet

A unified reference guide for using **Git Bash** and **Windows PowerShell** efficiently during development, data analysis, and reproducible research projects.

Navigation Commands

Task	Git Bash Command	PowerShell Command
Show current directory	<code>pwd</code>	<code>Get-Location</code>
List files and folders	<code>ls</code>	<code>Get-ChildItem</code> or <code>ls</code>
Change directory	<code>cd <path></code>	<code>Set-Location <path></code> or <code>cd <path></code>
Create a directory	<code>mkdir <folder></code>	<code>New-Item -ItemType Directory -Name <folder></code>
Remove a file	<code>rm <file></code>	<code>Remove-Item <file></code>
Remove a directory	<code>rm -r <folder></code>	<code>Remove-Item <folder> -Recurse</code>
Clear the screen	<code>clear</code>	<code>Clear-Host</code> or <code>cls</code>

File Management

Task	Git Bash Command	PowerShell Command
Create an empty file	<code>touch <file></code>	<code>New-Item <file></code>
Copy a file	<code>cp <source> <destination></code>	<code>Copy-Item <source> <destination></code>
Move or rename a file	<code>mv <source> <destination></code>	<code>Move-Item <source> <destination></code>
View file contents	<code>cat <file></code>	<code>Get-Content <file></code>
Search text inside files	<code>grep <pattern> <file></code>	<code>Select-String <pattern> <file></code>

Git Version Control

Task	Git Command	Description
Initialize repo	<code>git init</code>	Create a new local repository.
Clone repo	<code>git clone <url></code>	Copy a repository from GitHub or another remote.
Check status	<code>git status</code>	Show modified, staged, and untracked files.
Stage files	<code>git add <file></code>	Stage a file for the next commit.
Stage all files	<code>git add .</code>	Add all modified/untracked files.
Commit changes	<code>git commit -m "message"</code>	Save staged changes with a description.
View history	<code>git log</code>	Show the commit history.
Create new branch	<code>git branch <name></code>	Create a new branch.
Switch branches	<code>git checkout <name></code>	Switch to another branch.
Merge branches	<code>git merge <branch></code>	Merge a branch into the current one.
Pull changes	<code>git pull origin main</code>	Update your local repo from the remote.
Push changes	<code>git push origin main</code>	Upload commits to remote repository.
Undo last commit (keep changes)	<code>git reset --soft HEAD~1</code>	Undo last commit but keep file changes staged.
Undo commit and discard changes	<code>git reset --hard HEAD~1</code>	Undo last commit and discard changes.
Create <code>.gitignore</code>	<code>echo "*.csv"</code> <code>>> .gitignore</code>	Add file types to ignore list.

Project and Script Execution

Task	Git Bash	PowerShell
Run R script	<code>Rscript script.R</code>	<code>Rscript .\script.R</code>
Run Python script	<code>python script.py</code>	<code>python .\script.py</code>
Run Makefile	<code>make all</code>	<code>make all</code> (if Make installed)

Task	Git Bash	PowerShell
List environment variables	<code>env</code>	<code>Get-ChildItem Env:</code>
Print one environment variable	<code>echo \$VAR</code>	<code>\$Env:VAR</code>

Networking and System Info

Task	Git Bash Command	PowerShell Command
Check internet connection	<code>ping google.com</code>	<code>Test-Connection google.com</code>
Show running processes	<code>ps</code>	<code>Get-Process</code>
Show system info	<code>uname -a</code>	<code>Get-ComputerInfo</code>
Display date/time	<code>date</code>	<code>Get-Date</code>

Cleanup and Utilities

Task	Git Bash	PowerShell
Remove all .Rhistory files	<code>find . -name "*.Rhistory" -delete</code>	<code>Get-ChildItem -Path . -Recurse -Include *.Rhistory Remove-Item</code>
Delete empty folders	<code>find . -type d - empty -delete</code>	<code>Get-ChildItem -Directory -Recurse Where-Object { !(Get-ChildItem \$_.FullName) } Remove-Item</code>
Compress folder	<code>tar -czvf archive.tar.gz folder</code>	<code>Compress-Archive -Path folder - DestinationPath archive.zip</code>
Unzip file	<code>tar -xzvf archive.tar.gz</code>	<code>Expand-Archive archive.zip - DestinationPath . /</code>

Example Workflow (BIOS Project)

1. Clone your repo:

```
git clone https://github.com/yourname/bios512-project.git
```

2. Navigate to project:

```
cd bios512-project
```

3. Run the analysis Makefile:

```
make report.pdf
```

4. Commit and push results:

```
git add .  
git commit -m "Final project report and figures"  
git push origin main
```

Pro Tips

- Use `Ctrl + L` to clear your terminal in both Git Bash and PowerShell.
- Use **tab completion** to quickly complete file names or Git branches.
- Use `git diff` to preview changes before committing.
- Use PowerShell aliases (`ls`, `cat`, `rm`) for Linux-style commands.
- In Git Bash, you can combine commands with `&&` — e.g., `git add . && git commit -m "update"`.

This cheat sheet is ideal to include in a `/docs` folder of your GitHub repository for your BIOS 512/611 project, ensuring portability and clear workflow documentation.